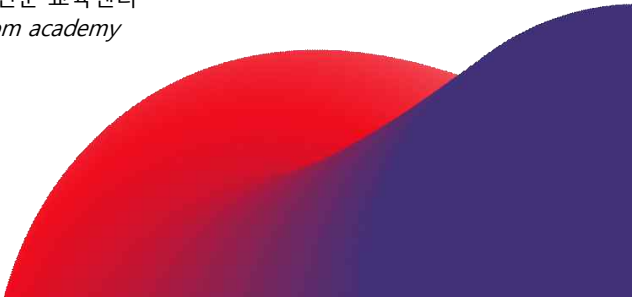


Algorithm in Python

"IT 융합 전문 교육센터"

Hancom academy



Copyright Note

- Issue Date: 2026.03.01
- Homepage: www.hancomacademy.com
- email: edu@hancom.com
- author: shin seungcheol
- Copyright© 2026 *Hancom innostream*, Co., Ltd. All rights reserved.
- 본 저작물의 저작권은 Hancom academy 및 저작권자에게 있으므로 저작권자의 사전 서면 허가 없이 무단으로 전재 혹은 복제를 금합니다.

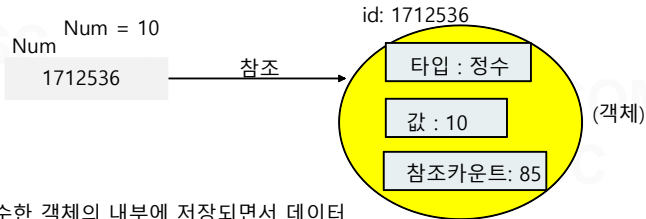
Chapter 01. Python Introduction

- 1. Data and Variable Types
- 2. if, Loops
- 3. Function
- 4. File I/O Operation 및 Control
- 5. Regular Expression

1. Data and Variable Types

1.1 파이썬 변수

- 동적 타이핑 언어로서 파이썬은 실행시간에 변수의 데이터 타입이 결정
따라서 타 언어 와 달리 변수 선언이 필요 없음
- 파이썬 변수 : 객체 id를 저장하는 변수 이며 id를 이용해 객체를 참조하는 개념



파이썬이 제공하는 특수한 객체의 내부에 저장되면서 데이터의 타입이 검사된후 타입에 대한 정보도 같이 저장

```
a = 3.14  
print(type(a))  
a = 8  
print(type(a))  
a = "python"  
print(type(a))
```



```
<class 'float'>  
<class 'int'>  
<class 'str'>
```

1.2 파이썬 객체 메서드

Python Code

```
String_object = "python programming"
```

```
total = [ ] # 빈 리스트
```

```
for x in string_object:
```

```
    if x == " ":
```

```
        total.append(" ")
```

```
    else:
```

```
        total.append(chr(ord(x)-32))
```

```
string_big = "".join(total)
```

```
print("for statement used :")
```

```
print(string_big)
```

```
print("=====")
```

```
print("upper method used :")
```

```
string_up = string_object.upper()
```

```
print(string_up)
```

리스트 객체 메소드

객체의 내장 메소드 활용 시 손쉽게
코드 구현 가능

문자열 객체 메소드

실행 결과

```
for statement used :  
PYTHON PROGRAMMING  
=====  
upper method used :  
PYTHON PROGRAMMING
```

1.3 파이썬 내부 데이터 Type

분류	내장 타입	타입명	상수
숫자	정수	int	520
	실수	Float	3.14
	복소수	Complex	5+2j
논리	참, 거짓	bool	True

```
a = 520  
print(type(a))  
a = 3.14  
print(type(a))  
a = 5+2j  
print(type(a))  
a = True  
print(type(a))
```



```
<class 'int'>  
<class 'float'>  
<class 'complex'>  
<class 'bool'>
```

1.3 파이썬 내부 데이터 Type

분류	내장 타입	타입명	상수
시퀀스	문자열	str	"programming"
	리스트	list	[1,2,3,4]
	튜플	tuple	(5,6,7)
셋	셋	Set	{2,4,6,8}
매핑	딕셔너리	dict	{"a":1, "b":2, "c":3}

```
a = "programming"
print(type(a))
a = [1,2,3,4]
print(type(a))
a = (5,6,7)
print(type(a))
a = {2,4,6,8}
print(type(a))
a = {"a":1, "b":2, "c":3}
print(type(a))
```



```
<class 'str'>
<class 'list'>
<class 'tuple'>
<class 'set'>
<class 'dict'>
```

1.4 시퀀스타입_문자열

Python Code

```
str1 = 'python'
print(str1)
str2 = "I'm python"
print(str2)
str3 = """Good python"""
print(str3)
str4 = """python
is
good programm"""
print(str4)
```



결과

```
python
I'm python
"Good python"
python
is
good programm
```

- 문자열 안에 큰 따옴표가 들어갈 경우 작은 따옴표로 문자열 생성
- 작은 따옴표가 있는 문자열은 큰 따옴표로 문자열 생성

• 이스케이프 시퀀스

기호	설명
\n	문자열에서 행(줄) 바꿈
\r	커서의 위치를 맨 앞으로 이동
\t	수평 탭 이동

SSC

1.5 문자열 기본 연산

```
str1 = "Good"  
str2 = "python programming"  
str3 = str1 + " " + str2  
print(str3)
```

→ 문자열 + 연산

```
stm = "Python_string "  
print(stm * 3)
```

→ 문자열 * 연산

```
sti = "string_idx"  
print(sti[0], sti[1], sti[len(sti)-1])
```

→ 문자열 색인 연산

```
print(sti[::-1])  
print(sti[::-1])
```

→ 문자열 길이 계산 후 마지막 인덱스 문자 출력

→ xdi_gnirts
→ string_idx

- 시퀀스 타입 인덱스(index) 색인은 항상 0 부터 시작

1.5 문자열 기본 연산

• in 연산

```
mystr = "Hello python  
programming"
```

```
print('pro' in mystr)
```

```
count = 0  
for s in mystr:  
    if s == 'o':  
        count += 1  
print(" o char count : %d" %count)
```

- 문자의 크기는 공백문자 < 대문자 < 소문자 순서
- 알파벳(사전) 순서로 크기가 증가

- 시퀀스 타입에 적용 가능한 in 연산
- 어떤 문자 또는 문자열이 있는지 확인 가능
- bool 타입 결과를 반환

• 문자열 비교 연산

```
print('a' < 'b')  
print("python" < "programming")  
print("py" < "python")
```



True
False
True

• 문자열 비교 예

```
saved_pw = 'python'  
  
while True: # 무한 반복 구문  
    input_str = input("password 입력( 종료 : quit) ==> ")  
    if input_str == saved_pw:  
        print('pw success!!')  
    elif input_str == 'quit':  
        break # 반복문 탈출  
    else:  
        print('pw fail!!')
```

1.6 문자열 포매팅

- 서식 지정자 이용한 문자열 포매팅

```
print("%s is %d age" %("Hong" , 50))
```

- %d, %x, %c, %s 등 다양한 서식 지정자가 있음

- format() 함수를 이용한 문자열 포매팅

```
mystr = "이름 {}, 나이 {}, 몸무게 {}".format("홍길동", 30, 68.5)  
print(mystr)
```

```
import time  
mynowtime = time.localtime()  
  
print("현재 시각은 {}년 {}월 {}일 {}시 입니다.".format(mynowtime.tm_year,  
mynowtime.tm_mon,mynowtime.tm_mday, mynowtime.tm_hour))
```

1.6 문자열 포매팅

- 고급 문자열 포매팅 - 실수 표현

```
fdata = 3.421356  
fdata2 = 5.345679  
print("{}".format(fdata))  
print("{0:.4f}".format(fdata))  
print("{0:.4f} or {1:.3f}".format(fdata, fdata2))
```

포매팅할 데이터 지정
idx

지정한 실수 데이터를 소수점
이하 3자리까지만 출력



```
3.421356  
3.4214  
3.4214 or 5.346
```

1.7 문자열 클래스 메소드

- `capitalize()` – 문자열의 첫 문자를 대문자로 변환

```
mystr = "python"  
print(mystr.capitalize())
```

- `center()` – 문자열의 형식에 맞게 중앙 정렬

```
menu_start = " menu titie "  
print(menu_start.center(30,'#'))
```



menu titie

- `count()` – 전달된 문자열 과 동일한 부분 문자의 개수 반환

```
mystr = """Time is like Gold.  
Time is like an arrow  
study Time is import."""  
print(mystr.count("Time"))
```



3

HANCOM
SSC

1.7 문자열 클래스 메소드

- `join()` – 인수로 전달된 시퀀스 객체의 각 항목 사이에 특정 문자를 삽입



```
mylist = ["Beautiful!", "Explicit!", "Simple!", "Complex!"]  
mystrjoin = '/'.join(mylist)  
print(mystrjoin)
```

➡ Beautiful!/Explicit!/Simple!/Complex!

- `split()` – 인수로 전달된 문자로 문자열을 분할하여 리스트 형태로 반환 // `join()` 메소드와 반대 개념



```
mysplitlist = mystrjoin.split('/')  
print(mysplitlist)
```

➡ ['Beautiful!', 'Explicit!', 'Simple!', 'Complex!']

```
myjoinstr = '\n'.join(mysplitlist)  
print(myjoinstr)
```



```
Beautiful!  
Explicit!  
Simple!  
Complex!
```

* `strip()` : 앞뒤 공백, `'\n'`, `'\r'` 제거

1.7 문자열 클래스 메소드

```
msg = "time is gold"
print(msg)
msg = msg.replace("gold", "arrow")
print(msg)
msg = msg.replace('i', '%')
print(msg)
```



```
time is gold
time is arrow
t%me %s arrow
[Finished in 0.4s]
```

- `replace()` 함수 : 문자열에 있는 특정 문자 또는 문자열을 다른 문자 또는 문자열로 바꿈

```
pin_number = "990415-1234112"
yymmdd = pin_number[:6]
print(yymmdd)
number_id = pin_number[7:]
print(number_id)
```



```
년월일 : 990415
Id : 1234112
[Finished in 1.3s]
```

```
if pin_number[7] == '1':
    print("남성")
else:
    print("여성")
```

1.8 시퀀스타입_리스트 기본 연산

- 시퀀스 타입이 갖는 공통된 특징으로 색인 연산 등 문자열에 사용된 연산이 같은 형태로 사용

```
mystr = "programming"  
print(mystr[1])  
mylist = ['p','r','o','g','r','a','o','m','i','n','g']  
print(mylist[1])
```

```
print(mystr+mystr)  
print(mylist+mylist)
```

```
print(mystr*2)  
print(mylist*2)
```

```
print(mystr[0:5])  
print(mylist[0:5])
```

```
print(mystr[0::2])  
print(mylist[0::2])
```

```
print(mystr[::-1])  
print(mylist[::-1])
```

```
print(list(mystr))  
print("".join(mylist))
```



색인 연산



+ 연산



* 연산

mylistdast1 = [None]*5



분할 연산



확장 분할 연산



역순 접근



문자열을 list 로



list를 문자열로

1.9 리스트(list) 특징

리스트만의 특징

- 문자열과 달리 리스트에 내포되는 객체들의 타입에는 제약이 없으며 객체라면 어떤 타입이든 리스트에 포함 가능

```
mylist = [ 3.14, 5, "python", [100,200,300], ["Good", "programming"]]
```

```
print(mylist[0])  
print(mylist[1])  
print(mylist[2][0])  
print(mylist[3][1])  
print(mylist[4][1])  
print(mylist[4][0][3])
```



```
3.14  
5  
p  
200  
programming  
d
```

- 실수, 정수, 문자열, 리스트 등 다양한 객체를 항목으로 내포 가능
- 내포된 객체에 대한 색인 연산

1.9 리스트 특징

```
mystr = "python"  
#mystr[0] = 'p'
```

immutable(불변) 객체
로서 수정 불가

```
mylist = ['p','y','t','h','o','n']  
mylist[0] = 'P'  
print(mylist)
```

mutable(가변) 객체
로서 수정 가능

```
mylistsrc = [ 3, 6, 9]  
mylistadd = mylistsrc + [20]  
  
print(mylistadd)  
print( "mylistsrc id :", id(mylistsrc) )  
print( "mylistadd id :", id(mylistadd) )
```

새로운 객체 생성

[3, 6, 9, 20]
mylistsrc id : 78857680
mylistadd id : 79218248



mutable 객체로서 리스트 항목을 직접 수정하기 위해서는
+=(복합할당연산), append(), extend(), insert() 메소드 활용

1.10 리스트 항목 추가 메서드

리스트 항목 추가 방법

```
mylistsrc = [ 3, 6, 9]
print(mylistsrc)
print("mylistsrc id :", id(mylistsrc))
mylistsrc.extend([200,300])
print(mylistsrc)
print("mylistsrc id :", id(mylistsrc))
```



```
[3, 6, 9]
mylistsrc id : 60310992
[3, 6, 9, 200, 300]
mylistsrc id : 60310992
```

extend() 메소드 활용

- **extend 메소드** : 전달된 객체(iterable)를 맨 마지막에 하나씩 풀어서 추가
전달된 객체는 iterable 객체여야 하며 +=(복합 할당 연산자)와 동일한 동작

```
mylistsrc = [ 3, 6, 9]
print(mylistsrc)
print("mylistsrc id :", id(mylistsrc))
mylistsrc.insert(len(mylistsrc),[200,300])
print(mylistsrc)
print("mylistsrc id :", id(mylistsrc))
```



insert() 메소드 활용

```
[3, 6, 9]
mylistsrc id : 3163600
[3, 6, 9, [200, 300]]
mylistsrc id : 3163600
```

- **insert 메소드** : append()메소드 와 동일 동작이지만 **추가될 위치를 지정할 수 있음**

1.11 리스트 항목 삭제 메서드

리스트 항목 삭제

```
mylist = [1,2,3,4,5]
```

```
print(mylist)  
del mylist[0]  
print(mylist)
```

→ del 키워드 활용 (인덱스 지정 삭제)

```
mylist.remove(7)  
print(mylist)
```

→ remove 메소드 활용 (삭제할 값 지정)
: 동일한 값이 여러 개 일 경우 맨 앞쪽 값만 삭제

```
deldata = mylist.pop()  
print(deldata)  
print(mylist)
```

→ pop 메소드 활용 (삭제 값 반환)
: 기본 적으로 맨 마지막 값 삭제



[1, 2, 3, 4, 5]

[2, 3, 4, 5]

[2, 4, 5]

5

[2, 4]

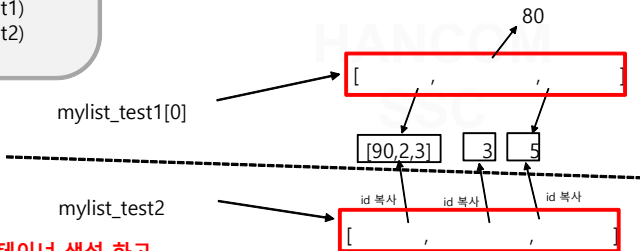
1.12 리스트 복사

- 리스트 복사 활용 예제 (얕은 복사 : `copy`)

```
import copy
mylist_test1 = [ [1,2,3], 3, 5]
mylist_test2 = copy.copy(mylist_test1)
print(mylist_test1)
print(id(mylist_test1[0]))
mylist_test2[0].append(99)

print("mylist_test1 : ", mylist_test1)
print("mylist_test2 : ", mylist_test2)
print(id(mylist_test2[0]))
```

얕은 복사 :
참조 id를 복사



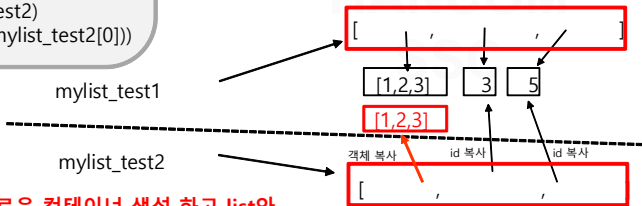
- `copy` 함수 호출 이후 새로운 컨테이너 생성 하고
리스트 항목 ID를 복사

1.12 리스트 복사

• 리스트 복사 활용 예제 (깊은 복사 : `deepcopy`)

```
import copy
mylist_test1 = [ [1,2,3], 3, 5]
print("mylist_test1 Id:", id(mylist_test1))
mylist_test2 = copy.deepcopy(mylist_test1)
print(mylist_test1)
print("mylist_test2 Id:", id(mylist_test2))
print("mylist_test1[0] id :", id(mylist_test1[0]))
mylist_test1[0].append(99)
print("mylist_test1 : ", mylist_test1)
print("mylist_test2 : ", mylist_test2)
print("mylist_test2[0] id :", id(mylist_test2[0]))
```

깊은 복사 :
list와 같은 mutable한 객체는 객체에 대한 복사



- `deepcopy` 함수 호출 이후 새로운 컨테이너 생성 하고 list와 같은 mutable 객체는 객체 자체 복사

1.13 리스트 정렬

- 리스트 정렬 (메소드)

```
mylistdata = [4, 5, 8, 2, 9]
mylistdata.sort()
print(mylistdata)
mylistdata.reverse()
print(mylistdata)
```

항목의 오름차순 정렬 메소드

정렬된 항목을 반대 저장

- 리스트 정렬 (내장 함수)

```
mylistdata = [4, 5, 8, 2, 9]
mylistdata2 = sorted(mylistdata)
print(mylistdata)
print(mylistdata2)
```

정렬된 리스트 반환

기존 리스트 변화 없음

HANCOM
SSC

- Join 메소드 리스트 활용

```
mylistdata = [ "Time ", "is ", "Gold" ]

res = "".join(mylistdata)
print(res)
```

1.14 리스트 활용 예제

• 리스트 활용 예제

```
myscorelist = [ ["math", 89] , ["english"]]  
print(myscorelist)
```

```
myscorelist[1].append(95)  
print(myscorelist)
```

```
myscorelist.append(["korean"])  
print(myscorelist)
```

```
myscorelist[2].append(79)  
print(myscorelist)
```

```
sum_data = 0  
for data in myscorelist:  
    sum_data += data[1]
```

```
print(sum_data)  
avg = sum_data / 3  
print("avg : {0:0.4f}".format(avg))
```



```
[['math', 89], ['english']]  
[['math', 89], ['english', 95]]  
[['math', 89], ['english', 95],  
 ['korean']]  
[['math', 89], ['english', 95],  
 ['korean', 79]]  
263  
avg : 87.6667
```

• 리스트 type 변환

```
mystr = "python"  
myconv_tuple = tuple(mystr)  
print(myconv_tuple)  
mylist = list(myconv_tuple)  
print(mylist)  
myconv_str = "".join(mylist)  
print(myconv_str)
```


1.14 리스트 활용 예제

• 리스트 활용 예제

```
mystr = 'mbc, kbs, sbs, jtbc '  
listdata = [ x.strip() for x in mystr.split(',') ]  
print(listdata)
```

```
strlist = ['foo','bar','barz']  
  
mylist = []  
for i, x in enumerate(strlist):  
    mylist.append((x,i))  
print(mylist)
```

• 리스트 join, split 활용 예제

```
mylist = [ 3.24, 2.22, 0.12 ]  
print(mylist)  
print(id(mylist))  
mystr = '/'.join( [str(i) for i in mylist] )  
print(mystr)  
mylist = mystr.split('/')  
print(mylist)  
print(id(mylist))
```

반복 가능한 객체(iterable)
의 각 항목에 순번 부여

1.15 튜플(tuple)

- 튜플 생성 및 연산 # 콤마(,)로 구분지어 튜플 생성 해야 함
: 리스트와 달리 **immutable** 객체로 생성된 후 내용 수정 불가

```
mydata1 = (1)
print(type(mydata1))
mydata2 = (1,)
```

→ 괄호 연산으로 인식
→ <class 'int'>
→ <class 'tuple'>

- 시퀀스 타입 공통 특성인 색인 연산, 덧셈 연산, 곱셈 연산, 분할 연산, 확장분할 연산 그대로 적용

```
mytuple1 = (3,6,9)
print(mytuple1[0])
mytuple2 = (12,15,18)
print(mytuple1 + mytuple2)
print(mytuple2[0:2])
print(mytuple2[:: -1])
```

```
# mytuple2[0] = 5
# del mytuple2[0]
```



immutable 객체로 내용 수정
삭제 불가

1.15 튜플(tuple)

• 튜플 활용 예제

```
def func1(mytuple_arg):  
    sum_data = 0  
    for x in mytuple_arg:  
        sum_data += x  
    avg = sum_data / len(mytuple_arg)  
    return sum_data, avg
```

```
mydata = (90, 71, 83, 63, 85, 63, 89)  
total_avg = func1(mydata)
```

```
print(type(total_avg))  
print(total_avg) # ( 총합, 평균 )
```

• 튜플 format 출력

```
mylist = [123, 567, 89]  
mytuple = ("127.0.0.1", 66578)  
print("client{} 가 접속".format(mytuple))  
print("{} 전송 데이터".format(mylist))
```

• 튜플 덧셈 연산

```
mytupledata = (1, 2, 3)  
print(mytupledata)  
print(id(mytupledata))  
mytupledata = mytupledata + (5,)  
print(mytupledata)  
print(id(mytupledata))
```



- 리스트와 달리 immutable 특징으로append, extend, insert 메소드 없음
- 덧셈 연산으로 새로운 객체 생성

1.15 튜플(tuple)

- 튜플 활용 예제 : 대소문자가 섞인 이름 데이터를 소문자로 변환 후 정렬

```
NameData = ("Lee", "park", "Hong", "shin")
```

```
def lowerModify(Name_arg):  
    mylist = []  
    for x in Name_arg:  
        mylist.append(x.lower())  
    return mylist
```

```
Namelowerlist = lowerModify(NameData)  
sortedData = sorted(Namelowerlist)  
print(tuple(sortedData))
```

→ 튜플은 데이터 추가, 삭제, 수정이 안됨으로 소문자로 변환 데이터를 리스트로 변환 저장

```
mytuple = ( "1.Insert" , "2.Display" , "3.Delete" , "4.Exit" )
```

```
for x in mytuple:  
    print(x, " ", end="")  
else:  
    print('\n')
```

→ 많은 문자열 객체를 묶어서 읽기 전용 처리 시 용이

HANCOM
SSC

1.16 사전(dict)

- 사전 생성 : { key : value } 형태

```
mydict = {}  
print(type(mydict))
```

빈 사전 생성

<class 'dict'>

- 사전 특징

```
mydict["Kor"] = 100  
mydict["Math"] = 90  
mydict["Eng"] = 80
```

Key 와 Value 형태로 빈
사전에 추가

```
print(mydict)  
print(mydict["Kor"])  
mydict["Kor"] = 88  
print(mydict)
```

Key 색인 연산으로 Value 출력

Key 색인 연산으로 Value 수정

- key(키) 는 변경 불가능한 객체(immutable 객체) 여야 하며 동일한 키 값이 존재 하지 않는 고유한 값이 여야 함
- 시퀀스 타입과 달리 순서가 지켜지지 않으며 순서가 없기 때문에 key(키)로 사전 객체 접근

1.16 사전(dict)

• 사전 내용 추가

```
mydict_test = {}  
  
mydict_test["A"] = 0  
mydict_test["B"] = 1  
mydict_test["C"] = 2
```

```
print(mydict_test)
```

Key 값과 value를 이용해
사전 내용 추가 가능



• list 내용 추가

```
mylist = [ ]  
mylist[0] = 5  
mylist[1] = 7  
print(mylist)
```

IndexError: list
assignment index out
of range



```
mylist = []  
mylist.append(1)  
mylist.append(2)  
print(mylist)
```

- list는 append, extend,
insert 메소드를 활용해
내용 추가

```
a = 77  
b = [1,2,3]  
c = "python"
```

```
dict = { a:b, tuple(b):c, c:a }  
print(dict)
```



사전의 key 값은 변경 불
가능한 immutable 객체
여야 함

1.16 사전(dict)

• 사전 내용 삭제

```
dict_data = { 'A':90, 'B':80, 'C':90 }  
res = dict_data.pop('B')  
print(dict_data)  
print(res)
```

```
dict_data = { 'A':90, 'B':80, 'C':90 }  
del dict_data['B']  
print(dict_data)
```

• 사전 내용 추가 및 변경

```
people_dict = { 'china':13567, 'india':123456, 'america':98345 }  
print(people_dict['china'])  
people_dict['china'] = 56728  
print(people_dict['china'])  
people_dict['korea'] = 3267892  
print(people_dict)
```

• 사전 update 메소드 // 동일한 키가 존재하면 항목 수정, 동일한 키가 없다면 항목 추가

```
dict_test = { 'first':7, 'second':77, 'third':777 }  
modify_dict = { 'first':0, 'end':100 }
```

```
# dict_test.update(modify_dict)  
print(dict_test)
```

1.17 사전(dict) 활용 예

• 사전 활용 예제

```
def runprogram():
    while True:
        selnum = MenuSel()
        if selnum == 1:
            InsertData()
        elif selnum == 2:
            DisplayData()
        elif selnum == 3:
            DeleteData()
        elif selnum == 4:
            break

if __name__ == "__main__":
    addresslist = {}
    runprogram( )
```

```
meun sel : 1
이름 입력 : Lee
전번 입력 : 5555
1.Insert 2.Display 3.Delete 4.Exit

meun sel : 2
Shin : 12344
Kim : 9898
Lee : 5555
1.Insert 2.Display 3.Delete 4.Exit

meun sel : 3
삭제할 이름 입력 : Lee
1.Insert 2.Display 3.Delete 4.Exit

meun sel : 2
Shin : 12344
Kim : 9898
1.Insert 2.Display 3.Delete 4.Exit

meun sel : 4
```

* 위 동작이 되도록 각각 함수 기능 구현 / dict_addresslist.py

1.18 셋(set)

- Set 생성 : { 2,3,4 } 형태

test = {} # 빈 중괄호는 사전

```
myset = {1,2,3}  
print(type(myset))
```

→ 셋 생성
→ <class 'set'>

- Set(셋) 특징 : 중복을 허용하지 않음, 항목간의 순서 없음

```
mysetdata = { 1,1,1,1,3,3,4,5,5,6,6}  
print(mysetdata)
```



{1, 3, 4, 5, 6}

- Set 타입 항목 : 변경 불가능한 객체여야 함

```
mysetdata1 = { [1,2], 6 }  
print(mysetdata1)
```



```
mysetdata1 = { [1,2], 6 }  
TypeError: unhashable type: 'list'
```

ANCOM
SSC

- Set 타입 연산 : 집합 연산 처럼 합집합, 교집합, 차집합, 여집합 가능

```
mysetdata1 = { 3, 4, 5, 6}  
mysetdata2 = { 5, 6, 7, 8}  
res = mysetdata1 | mysetdata2 # 합집합  
print("합집합 : ", res)  
res = mysetdata1 & mysetdata2 # 교집합  
print("교집합 : ", res)  
res = mysetdata1 - mysetdata2 # 차집합  
print("차집합 : ", res)  
res = mysetdata1 ^ mysetdata2 # 교집합의 여집합  
print("교집합의 여집합 : ", res)
```

1.18 셋(set) 활용

```
mylist = [9,5,3,7,2,1,55, 32, 21, 99,3,9,6,7,6,7,4,1]
```

```
myset = set(mylist)  
print(myset)
```

→ 리스트를 set으로 중복제거

```
mylist_mod = list(myset)  
print(mylist_mod)
```

→ set을 리스트로

```
mylist_sort = sorted(mylist_mod)  
print(mylist_sort)
```

→ 리스트 항목
정렬

• 세 문자열에서 공통된 문자만 출력

```
mystr1 = "python is simple"  
mystr2 = "apple is delicious"  
mystr3 = "programming"
```

```
myset1 = set(mystr1)  
myset2 = set(mystr2)  
myset3 = set(mystr3)
```

```
myset_res = myset1 & myset2 & myset3  
print(myset_res)
```

HANCOM
SSC

2. if(제어문) / Loops(반복문)

2.1 제어문

● 단일 제어문

if 조건식:
수행 코드블럭



- 조건식이 참일때 코드블럭 수행
- 조건식이 거짓일때 코드블럭 수행 되지 않음

● 여러 경우의 조건을 판단하여 참 인 경우의 코드 블록 만 수행

if 조건식:
수행 코드블럭
elif 조건식:
수행 코드블럭
elif 조건식:
수행 코드블럭
else:
수행 코드블럭



- 조건식이 참인 코드블럭을 수행하고 if~elif~else 구문 전체를 탈출
- else: 조건식이 올수 없고 위 if~elif 구문의 조건식이 하나도 참이 없을 경우 else: 구문의 코드 블록을 수행

2.2 조건 표현식

```
a = 50; b=70  
Max_val = a if a>b else b  
print(Max_val)
```

if else 문을 한 줄로 간결히 표현
A표현식 if 조건식 else B표현식
조건식이 참이면 A표현식 수행
조건식이 거짓이면 B표현식 수행

• 리스트 내포

```
mylisttest = [ x for x in range(1, 10) ]  
print(mylisttest)
```

리스트 내포
[표현식 for x in 반복가능객체]
표현식을 리스트 항목으로 해서 리스트 생성

• 조건 표현식 활용한 list 내포 확장

```
mylist = [ "짝" if x%2 == 0 else "홀" for x in range(1,10) ]  
print(mylist)
```

리스트 내포의 표현식을 조건 표현식으로 구현
반복객체의 항목이 홀수이면 "홀"로
반복객체의 항목이 짝수이면 "짝"으로 생성

2.2 조건 표현식

- list 내포 확장

```
mylist3 = [ 3, 6, 5, 9, 8, 2, 1]
mylist4 = [ "짝" if x%2 == 0 else "홀" for x in mylist3 ]
print(mylist4)
```

리스트 내포의 표현식을 조건 표현식으로 구현
반복객체의 항목이 홀수이면 "홀"로
반복객체의 항목이 짝수이면 "짝"으로 생성

- list 내포 확장 // 리스트 내포를 이용해 소문자로 변환

```
strarg = 'PYTHON PROGRAM'
```

```
print(''.join(listData))
```

python program

- 리스트 여과기

```
mylist2 = [ x for x in range(1,10) if x%2 == 0 ]
print(mylist2)
```

리스트 내포에 조건문을 추가하여 여과기 기능
반복객체 중 짝수인 값만 리스트 항목 추가
단, else 추가시 syntax 오류

2.3 Loop (반복문)

- 1~10까지 홀수의 합 계산

```
total = 0
for x in range(1,10):
    if x%2 == 0:
        continue # break
    total += x # total = total + x
print("1~10 까지 홀수 합 : ", total)
```

- "python" 문자열을 for문을 활용해 대문자로 변환 하되 'y' 문자만 소문자 그대로 표현

```
for ch in "python":
    if ch == 'y':
        print("{}".format(ch))
    else:
        print("{}".format( chr(ord(ch)-32) ) )
```

- 사전 { } 내용을 key 와 value 구분 하여 출력

```
mydic = { "a":1, "b":2, "c":3, "d":4 }

for dickey in mydic:
    print( "{", "}" : {} ".format(dickey, mydic[dickey], ")")
```

2.3 Loop (반복문)

- 사전 데이터 key, value를 서로 바꿔보기

```
my_dict = { 'a':1, 'b':2, 'c':3, 'd':4 }  
print(my_dict)
```

{'a': 1, 'b': 2, 'c': 3, 'd': 4}

```
print(rev_dict)
```

{1: 'a', 2: 'b', 3: 'c', 4: 'd'}

- 반지름을 입력 받아 원의 넓이 및 둘레를 계산, "end" 입력시 프로그램 종료

```
while True:
```

```
    minput = input("반지름 입력 : ")
```

```
    if minput == "end":
```

```
        break
```

```
    else:
```

```
        myradius = float(minput)
```

```
    print("myradius : ", myradius)
```

```
    circle_area = myradius * myradius * 3.14
```

```
    circle_len = 2 * 3.14 * myradius
```

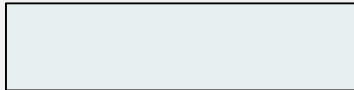
```
    print("area : ", circle_area)
```

```
    print("len : ", circle_len)
```

2.3 Loop (반복문)

- 문자열 데이터 중 숫자문자 나 특수 문자 제거한 영문대(소) 문자만 추출

```
mystrdata = "#s45cD!K2ab@"  
mylistdata = []
```



```
print(mylistdata)  
mystr_cov = "".join(mylistdata)  
print(mystr_cov) # scDKab
```

- list 내포를 활용 1~100까지 숫자 중 2와 3의 공배수 중 4의 배수가 아닌 수의 리스트 생성

```
mylistdata = [x for x in range(1,101) if ((x%2==0 and x%3==0) and (x%4 != 0)) ]  
print(mylistdata)
```

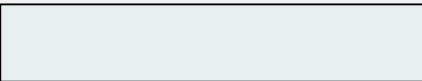
- list 내포를 활용 range(1,20) 을 순회하여 3으로 나누었을 때 나머지가 1 이면 'A' 나머지가 2이면 'B', 나머지가 0이면 'C'로 리스트 생성

```
mylistdata = [ 'A' if x%3 == 1 else 'B' if x%3 == 2 else 'C' for x in range(1,20) ]  
print(mylistdata)
```


2.3 Loop (반복문)

- 알파벳 순으로 단어 데이터 수집 및 취합

```
words = [ 'apple', 'bat', 'air', 'bar', 'atom', 'book' ]  
by_letter = { }
```



```
print(by_letter)
```

→ { 'a': ['apple', 'air', 'atom'], 'b': ['bat', 'bar', 'book'] }

- list 내포 활용 단어의 길이가 2 보다가 큰 단어만 대문자로 변환해서 리스트 생성

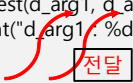
```
mystr = [ 'a', 'as', 'bat', 'car', 'dove', 'python' ]  
  
mylist = [ x.upper() for x in mystr if len(x) > 2 ]  
print(mylist)
```

3. Python Function(함수)

3.1 함수 설명

- 함수 : 특정 기능(동작)을 코드 블록으로 묶어서 수행 코드의 재활용성과 유지 보수성을 향상
- 함수 정의
`def` 함수명(형식인수, 형식인수, ...): # 파라미터
 코드 블록
- 함수 호출
 함수명(실인수 , 실인수, ...) # 함수 형식인수에 전달할 실 인수 값 사용

```
def myfunctest(d_arg1, d_arg2):  
    print("d_arg1 : %d, d_arg2 : %d" %(d_arg1, d_arg2))  
  
val = 70  
myfunctest(val , 50)
```



3.2 함수 인수

• 기본값을 가진 형식인수

1부터 실 인수 까지 중 짝수 만 출력

```
def evenprint( n, step = 1):  
    x = 1  
    while x <= n:  
        if x%2 == 0:  
            print(x)  
        x += step
```

```
evenprint(10)
```

```
print( end='\\n')
```

• 가변 인수 활용 (튜플, 사전)

```
def argsfunc(*args):  
    i = 0  
    for x in args:  
        i += 1  
    print("args 인수의 개수 : %d" %i)  
    print(args)  
    print(args[0])  
    print(args[1])  
    print(args[2])
```

```
def dictsfunc(**dicts):  
    i = 0  
    for x in dicts.keys():  
        i += 1  
    print("dicts 인수의 개수 : %d" %i)  
    print(dicts)
```

```
argsfunc(1,[2,4],{'a':1,'b':4,'c':5})  
dictsfunc(a=1,b=2,c=3)
```

3.3 함수 활용

- 튜플, 사전, 튜플 리스트 처리 함수

```
def func3(myarg):  
    for num, value in myarg:  
        print(num, value)  
  
def func2(myarg):  
    for num in myarg:  
        print(num, myarg[num])  
  
def func1(myarg):  
    for num in myarg:  
        print(num)  
  
mydata = (1,2,3,4,5)  
mydata2 = { 'A':1, 'B':2, 'C': 3}  
mydata3 = [ (1,2), (6,8), (9, 3)]  
  
func1(mydata)  
func2(mydata2)  
func3(mydata3)
```

- return 구문

```
def myfunctest(d_arg1, d_arg2):  
    res = d_arg1 + d_arg2  
    return res  
  
val = 70  
sum_d = myfunctest(val, 50)  
print("sum : ", sum_d)
```

- return 키워드 : 값의 반환 과 함수 종료 기능
- return 이 없다면 함수 종료 시 기본적으로 None 객체 반환

```
return val, tmp  
# (val,tmp) 튜플 형태로 반환함  
으로 여러 개의 값을 반환 할 수  
있음
```

3.3 함수 활용

- 가변 인수로 받은 데이터에 실 인수 값의 곱한 내용을 출력

```
def mysum_func( mul, *args):  
    cnt = 0  
    tuple_len = len(args)  
    while cnt < tuple_len:  
        print("{}".format(mul * args[cnt]))  
        cnt += 1
```

```
res = mysum_func(5, 1,2,3,4,5)
```

주의) 가변 인수는 위치인수보다
뒤에 위치 시켜야 함

- 알파벳 문자열을 실 인수로 전달하여 문자열을 모두 대문자로 변환하는 함수

```
def conv_string_upper(mystr):  
    convstr = mystr.upper()    // 대문자 변환 메소드  
    print(convstr)
```

```
conv_string_upper("python")
```

3.4 함수 스코핑 룰

- 스코핑룰

```
gdata = 8  
def functiontest():  
    gdata = 50  
    print("gdata : ", gdata)  
functiontest()
```

함수 내 지역공간 gdata 변수 등록 사용으로
전역변수를 가려 버림

```
gdata = 8  
def functiontest():  
    print("gdata : ", gdata)  
functiontest()
```

지역 영역의 이름공간에
변수가 없다면 전역 이름공간에 선언된 변수 사용

- 파이썬 스코핑룰은 다음 규칙으로 변수를 찾아 사용

지역 이름공간



전역 이름공간



내장 이름공간

3.4 함수 스코핑 룰

- 스코핑 룰 에 따른 주의

```
gdata = 8
```

```
def functiontest():
```

```
    gdata += 50
```

```
    print("gdata : ", gdata)
```

```
functiontest()
```

UnboundLocalError: local variable 'gdata'
referenced before assignment

지역변수 등록 과정에서 초
기화되지 않은 지역변수 사
용 오류



전역변수 값 수정하려면 glob
키워드 사용

```
gdata = 8
```

```
def functiontest():
```

```
    global gdata
```

```
    gdata += 50
```

```
    print("gdata : ", gdata)
```

```
    print(locals())
```

```
functiontest()
```

```
print(gdata)
```

```
print(locals())
```

```
gdata : 58
```

```
{
```

```
58
```

```
{'__name__': '__main__', .....생략
```

```
'gdata': 58, 'functiontest': <function
```

```
functiontest at 0x00C90D68>}
```

등록된 지역
변수 없음

등록된 전역 변수
사용

3.5 함수 실전 예

- 전역변수 값을 swap 하는 함수 구현

```
a = 10
b = 20
def swap_func():
    global a, b
    a, b = b, a

print("a : %d, b : %d" %(a,b))
swap_func()
print("a : %d, b : %d" %(a,b))
```

- 문자열 데이터 임의의 문자열로 변환해 암호화 구현

```
def encrypt(msg):
    for ch in msg:
        if ch in encbook:
            msg = msg.replace(ch, encbook[ch])
    return msg

encbook = { 'p': '%', 'y': '(', 't': '#', 'h': '=', 'o': '@', 'n': '!' }

res_msg = encrypt("I love python programming")
print(res_msg)

dec_msg = Decryptfnc(res_msg)
print(dec_msg)
```

사전에 있는 key 와
일치하는 value 값으
로 변환

I l@ve %(#=@! %r@grammi!g
[Finished in 0.3s]

4. 파일 입출력 및 조작

4.1 파일 입출력 기본 사용법

```
f = open("Testfile1.txt", 'r') # 파일을 읽어와 파일 객체로 메모리에 로딩
for line in f:                 # 파일 객체 줄 단위로 항목이 순회
    print(line, end="")        # == print(line.strip())
else:
    print('\n')

f.seek(0,0)                    # 파일포인터 위치를 맨 앞으로
print(f.readlines())           # 파일의 내용을 리스트로 반환
f.close()                      # 파일 객체 close
```

결과 :

```
python programming
good
study
very very hot language
```

```
['python programming\n', 'good\n', 'study\n', 'very very hot language']
```

4.2 파일 입출력 with문 사용법 및 열기 모드

with open("Testfile1.txt", 'r') **as** f:

```
#print(f.read())    # 파일의 내용 전체를 문자열로 리턴  
#print(f.readlines()) # 파일의 모든 줄을 읽어서 각각의 줄을 요소로 가지는 리스트를 리턴  
print(f.readline())  # 파일의 첫 번째 줄을 읽어 리턴
```

- 기본적으로 사용하는 함수를 with문 안에 사용하게 되며 with as 구문을 나을 때 close를 자동으로 불러줍니다.

• 파일 오픈 모드

r : 읽기 모드, 파일을 읽을 때 사용합니다.
w : 기존 파일에 있던 데이터를 완전 지우고 새로 내용을 입력합니다.
a : 기존 파일에 있던 데이터를 그대로 두고 마지막에 내용을 입력합니다.
r+ : 기존 파일에 있던 데이터를 그대로 두고 그 위에 내용을 덮어씁니다.

4.3 파일 r+ 모드

```
f = open("Testfile1.txt", 'r+')
print(f.read())           # 파일 내용 전체 읽기
print(f.tell())          # 현재 파일 포인터 위치 출력
f.seek(0,0)              # 파일 포인터 위치를 맨 앞으로 이동

f.write("c/c++")          # 현재 위치에 "c/c++" 문자열 저장
f.flush()               # 쓰기 버퍼 비워주는 동작(실제 파일에 저장)
f.seek(0,0)
print(f.read())
f.close()
```



```
python programming
good
study
very very hot language
55
c/c++n programming
good
study
very very hot language
[Finished in 0.2s]
```

4.4 파일 입출력 활용 예

• 파일 내용 중 단어 구별

```
f = open("mytext.txt", "r") # "mytext.txt" == sys.argv[1]

mystr = f.read()

mylist = mystr.split(" ")

print(mylist)
mylist_cnv = {}
for s in mylist:
    tmp = s.strip()
    if tmp in mylist_cnv:
        mylist_cnv[tmp] = mylist_cnv[tmp] + 1
    else:
        mylist_cnv[tmp] = 1

print(len(mylist_cnv))
print(mylist_cnv)
```

읽어 들인 파일 내용을 공백(" ")
문자로 문자열 구분하여 list

단어에 포함된 \n or 공백 제거

단어별로 사전에 포함시키되 사전에
이미 단어가 있으면 사전 value 값 1
증가

4.5 파일 / 디렉토리 조작하기

```
import os
```

1) 현재 작업 폴더 얻기

```
==> print(os.getcwd())
```

2) 디렉토리 변경 (chdir)

```
==> os.chdir('C:/temp')
```

```
print(os.getcwd()) # C:\temp
```

```
print(os.listdir()) # ['AUtempR', 'NEO2'] <== temp 디렉토리 내부 디렉토리
```

3) 디렉토리 안의 파일/서브디렉토리 리스트 (**listdir**)

```
print(os.listdir()) # 현재 디렉토리 내부의 모든 디렉토리 및 파일 리스트 조회
```

```
print(os.listdir('c:/dist/')) # 특정 디렉토리 내부의 모든 디렉토리 및 파일 리스트 조회
```

◎ listdir() 과 유사기능의 함수(glob.glob)

```
import glob
```

```
folder = 'C:\Wdist'
```

```
file_list = glob.glob(f"{folder}W*")
```

```
# file_list = glob.glob(f"{folder}Wdet*.xlsx") # 특정이름 파일 및 확장자 파일만 필터링해서 추출
```

```
# 주의: 폴더(디렉토리) 경로 뒤 모든것을 의미하는 *(와일드 문자) 있어야함
```

```
for f in file_list:
```

```
    print(f)
```

```
# 출력 : 해당 파일및 폴더의 경로까지 추출
```

```
C:\Wdist\data
```

```
C:\Wdist\Wdetect.exe
```

```
C:\Wdist\Wdetect_category.xlsx
```

```
C:\Wdist\Wlog_info.txt
```

```
C:\Wdist\Wruns
```

```
C:\Wdist\Wolov5s.pt
```

◎ glog.glog() 처럼 `listdir()` 사용

```
import os
root_path = 'c:/dist'
filepath = os.listdir(root_path)
# 해당 디렉토리 내부의 모든 파일/디렉토리명 리스트로 반환 (경로 미 포함)
```

```
print(filepath)
# 출력
['data', 'detect.exe', 'detect_catogory.xlsx', 'runs', 'yolov5s.pt']
```

`endswith` : 문자열 함수 중 하나로 현재 문자열이 사용자가 지정하는 특정 문자로 끝나는지 체크하는 함수

```
pylist = [os.path.join(root_path, file) for file in filepath if file.endswith('.py')]
# 특정 확장자 파일만 필터링해서 추출
```

```
print(pylist) # 경로까지 조합해서 출력
```

4) 파일의 이름 바꾸기 (rename)

```
root_path = 'c:/dist/'
```

```
os.rename(root_path+'detect_test.txt',root_path+'detect_cfg.txt' )
```

5) 파일 지우기 (remove)

```
os.remove(root_path+'detect_cfg.txt' )
```

6) 디렉토리 생성(만들기) (**mkdir, makedirs**)

```
root_path = 'c:/dist/'
```

```
os.mkdir(root_path + 'test1') # 한 폴더(디렉토리)만 생성 가능
```

test1폴더 내부 test2폴더 생성, test2폴더 내부 test3폴더 생성 처럼 원하는 만큼 폴더를 생성할수 있음

exist ok = True 설정시 이미 기존 폴더 존재 할시 에러 없이 넘어가고, 없을 경우만 생성

```
os.makedirs(root_path + 'test1/test2/test3', exist_ok=True)
```


7) 디렉토리 지우기 (rmdir) : 내부가 비워있는 빈 디렉토리 삭제

```
os.rmdir(root_path+'test')
```

주의 :해당 폴더가 비워 있지 않을 경우 오류 발생,

해당 폴더의 하위 폴더 및 파일 모두 삭제 시 아래 shutil.rmtree 사용

```
import shutil
```

```
shutil.rmtree(root_path + 'test')
```

8) 경로 중 디렉토리명만 얻기 (dirname)

```
root_path = 'c:/dist/'
```

```
print(os.path.dirname(root_path + 'log_info.txt')) # c:/dist
```

9) 경로 중 파일명만 얻기 (**basename**)

```
root_path = 'c:/dist/'
```

```
print(os.path.basename(root_path + 'log_info.txt')) # log_info.txt
```

10) 경로 중 디렉토리명과 파일명 분리해서 얻기(**split**)

```
dir, file = os.path.split(root_path + 'log_info.txt')
```

튜플 반환 -> unpack

```
print(dir, file) # c:/dist log_info.txt
```

11) 경로를 병합하여 새 경로 생성 (**join**)

```
print(os.path.join(root_path, 'log_info.txt')) # c:/dist/log_info.txt
```

12) 파일 혹은 디렉토리 경로가 존재하는지 체크 (**exists**)

```
root_path = 'c:/dist/'  
print(os.path.exists(root_path + 'log_info.txt')) # True  
print(os.path.exists(root_path)) # True
```

13) 해당 경로에 특정 디렉토리 존재하는지 체크 (**isdir**)

```
root_path = 'c:/dist/'  
print(os.path.isdir(root_path + 'runs')) # True , runs : 디렉토리
```

14) 해당 경로에 특정 파일 존재하는지 체크 (**isfile**)

```
root_path = 'c:/dist/'  
print(os.path.isfile(root_path + 'log_info.txt')) # True
```

4.6 파일 / 디렉토리 조작(shutil 모듈)

```
import shutil
```

shutil 모듈은 파일 및 디렉토리 작업에 유용한 많은 기능을 제공.
주요 기능으로는 파일 복사, 이동 및 삭제, 디렉토리 복사와 제거, 압축 파일 처리 등이 있음.

➤ 파일 복사

shutil.copy(원본파일, 복사할 파일)

shutil의 copy 방법(메서드) 3가지

- copyfile / copy / copy2

copy/copyfile ==> 메타정보 복사 되지 않고 그냥 파일 복사

copyfile ==> 목적지가 디렉토리가 될수 없음, 정확히 복사할 경로와 파일명 기입, 파일만 복사

copy ==> 목적지가 디렉토리 올수 있음, 목적지 디렉토리에 파일을 복사해줌,

파일 권한까지복사

copy2 ==> 작성한날짜를 포함한 모든 메타정보까지 모두 복사

copyfile < copy < copy2 오른쪽으로 갈수록 포괄적인 개념

➤ 파일 복사

```
import os
import shutil
root_path = 'c:/dist/'
if os.path.exists(root_path+'log_info.txt'): # 'runs' 디렉토리 아래에 'log_info.txt'파일 복사
    shutil.copy2(root_path+'log_info.txt',root_path+'runs')
```

➤ 파일 이동

기존 또는 새파일명으로 파일 이동(복사와 다른 개념)

```
import os
import shutil
root_path = 'c:/dist/'

# shutil.move('원본파일.txt', '이동할폴더/새파일.txt')
if os.path.exists(root_path+'log_info.txt'): # 새파일명으로 이동
    shutil.move(root_path+'log_info.txt',root_path+'runs/'+ 'new_log.txt')

if os.path.exists(root_path+'log_info.txt'):
    shutil.move(root_path+'log_info.txt',root_path+'runs/') # 기존파일명으로 이동
```

➤ 파일 삭제 ==> `os.remove()` 활용

➤ 폴더 복사

파일을 포함하는 폴더 통째로 복사

`shutil.copytree('원본폴더', '새폴더')`

`shutil.copytree(root_path+'runs/'+ 'job', root_path+'runs/'+ 'good')`

폴더 삭제

폴더내 파일이 있어도 파일을 포함하는 폴더 통째로 삭제

`shutil.rmtree(root_path+'runs/'+ 'good')`

4.7 Python코드 실행 파일 만들기

arg_recv_exam.py

```
import sys
#print(len(sys.argv))
# argv[0] ==> 프로그램실행파일명
# argv[1] 부터 ~ ==> 실행파일에 전달하는 값
listdata = []
for item in sys.argv[1:]:
    listdata.append(int(item))

total = 0
for x in listdata:
    total += x
print('total : ', total)
```

input("Press any key to program exit...") # 사용자가 입력할 때까지 대기

==> pycharm 터미널 창 오픈
python .\arg_recv_exam.py 30 50 ↵ 명령 입력
결과
total : 80
Press any key to program exit...

* 시스템 인터프리터 사용시 실행 파일 생성 방법
PyInstaller 설치

`pip install pyinstaller`

콘솔창에 내용 출력되지 않도록 하기 (`-w` 옵션 또는 `--noconsole`)

실행파일 하나만 생성하기 (`-F` 옵션 또는 `--onefile`)

최종 예시)

`pyinstaller -w -F 파이썬소스.py`

`pyinstaller -F 파이썬소스.py`

* pycharm 가상 인터프리터 사용시 실행 파일 생성 방법

1) pycharm 터미널 오픈

2) `cd` 명령으로 해당 파일이 있는 위치로 이동

3) `python -m PyInstaller -w -F 파름.py` 입력

OR `python -m PyInstaller -F 파일이름.py` 일일이입력

이전 파이썬 코드 실행 파일 생성 명령

`python -m PyInstaller -F .Warg_recv_exam.py`

* arg_recv_exam.exe 실행 파일을 동작 시키는 GUI 프로그램

GUI_Sample.py

```
import tkinter as tk # 파이썬 기본 제공 GUI 라이브러리 모듈
```

```
import os
```

```
def bn1_clicked():  
    global incr  
    incr = incr+1  
    numbering.set(incr) # 증가된 값을 제어변수에 설정
```

```
def bn2_clicked():  
    global incr  
    incr = incr-1  
    numbering.set(incr) # 감소된 값을 제어변수에 설정
```



```
def bn3_clicked():  
    global incr  
    rootdir = os.getcwd() # 현재 경로 읽어옴  
    filename = os.listdir(rootdir)[0]  
    cmd = os.path.join(rootdir, filename) # 현재 경로 + 실행파일이름  
    cmd += '  
cmd += '5'  
cmd += '  
cmd += str(incr)  
print(cmd)  
os.system(cmd) # 실행파일이름 arglist를 command 창에 입력해 동작시키는 명령
```

```
incr = 0  
mywind = tk.Tk() # main 윈도우 객체 생성  
mywind.geometry('600x600')  
mywind.title('mypro')  
  
numbering = tk.IntVar() # 제어변수
```

```
bn1 = tk.Button(text='UP',padx=20,pady=20, background='blue',command= bn1_clicked) # 버튼  
자식 윈도우 객체 생성
```

```
bn1.pack() # 자식 윈도우 배치
```

```
bn2 = tk.Button(text='Down',padx=20,pady=20, background='red',command= bn2_clicked) # 버튼  
자식 윈도우 객체 생성
```

```
bn2.pack() # 자식 윈도우 배치
```

```
bn3 = tk.Button(text='현재값 전송',padx=20,pady=20, background='pink',command= bn3_clicked)  
# 버튼 자식 윈도우 객체 생성
```

```
bn3.pack() # 자식 윈도우 배치
```

```
lb1 = tk.Label(text='0', textvariable=numbering)
```

```
lb1.pack()
```

```
mywind.mainloop() # 화면에 윈도우 출력
```

GUI_Sample.py 코드 실행 파일 생성 명령

pycharm 터미널 오픈

```
python -m PyInstaller -w -F .\WGUI_Sample.py
```

5. 정규 표현식 (re 모듈)

: 파이썬은 정규표현식을 지원하기 위해 regular expression(re) 모듈을 제공함
(기본 라이브러리, 설치 필요 없음)

5.1 정규 표현식 설명

정규 표현식은 문자열에서 특정 패턴을 검색하거나 치환할 때 사용하는 표현식을 말함
다양한 문자열 객체 제어 메서드가 존재하지만 복잡한 패턴을 한 번에 처리할 수 있는 장점과
패턴 기반 작업으로 간결한 코드 작성에 용이함

- re.compile() 을 이용해서 검색 패턴 생성
- re.compile() 에 들어가는 식을 정규 표현식이라 함

import re 로 모듈을 사용하며,
p = re.compile('ab*')와 같이 정규표현식을 컴파일한다.
반환값은 p(컴파일된 패턴 객체)이다.

5.2 문자열 검색

- 컴파일된 패턴 객체를 사용해 문자열을 검색할 수 있다.
- 패턴객체가 제공하는 4가지 메서드

➤ **match()** :

문자열의 처음부터 검색하여 정규식과 매치되는 지 조사하고, 매치되면 match객체 반환,
비 매치시 None 반환

➤ **search()** :

문자열 전체를 검색하여 정규식과 매치되는 지 조사하고, 맨 처음 match된 match객체 반환,
비 매치시 None 반환

➤ **findall()** :

정규식과 매치되는 모든 문자열을 리스트로 반환

➤ **finditer()** :

정규식과 매치되는 모든 문자열을 반복가능한 객체로 반환

❖ match 메서드

- 문자열의 처음부터 정규식에 부합하는 지를 검사

```
import re
```

```
p = re.compile(r"[a-z]+")
```

```
m = p.match('python')
```

```
print(m) # <re.Match object; span=(0, 6), match='python'>
```

```
m = p.match('^^ python')
```

```
print(m) # 처음에 나오는 '^^'이 [a-z]+에 부합하지 않으므로 None을 반환
```

❖ search 메서드

- 문자열 전체를 검색하여 정규식에 부합하는 지를 검사

```
import re
```

```
p = re.compile(r"[a-z]+")
```

```
m = p.search('python prog')
```

```
print(m) # <re.Match object; span=(0, 6), match='python'>
```

```
m = p.search('1991 python progrmming')
```

```
print(m) # <re.Match object; span=(5, 11), match='python'> # 맨처음 매칭된 문자열
```

❖ findall 메서드

- 정규식과 매치되는 모든 문자열을 리스트로 반환

```
result = re.findall(r"[a-z]+", "파이썬으로 regular expression 배우기")  
print(result) # ['regular', 'expression']
```

문자열에서 [a-z]+에 부합하는 문자열을 리스트로 반환

❖ finditer 메서드

- 정규식과 매치되는 모든 문자열을 반복가능한 객체로 반환

```
result = re.finditer(r"[a-z]+", "파이썬으로 regular expression 배우기")  
print(result) # <callable_iterator object at 0x000002B5A0A4BC10>  
for r in result:  
    print(r) # r.group()으로 match 항목 추출가능
```

```
# <re.Match object; span=(6, 13), match='regular'>
```

```
# <re.Match object; span=(14, 24), match='expression'>
```

- match, search 객체가 가지는 메서드를 통해 인덱스와 문자열에 대한 정보를 확인

```
import re
```

```
result = re.search(r"[a-z]+",'1991 python programming')  
print(result) # <re.Match object; span=(5, 11), match='python'>
```

```
print(result.group()) # 처음 매치된 문자열 반환 : python  
print(result.start()) # 매치된 문자열의 시작 위치 반환  
print(result.end()) # 매치된 문자열의 끝 위치 반환  
print(result.span()) # 매치된 문자열의 (시작, 끝)에 해당하는 튜플 반환 : (5, 11)
```

SSC

5.3 정규표현식 특수기능 역할의 메타문자

❖ 메타문자 종류 : 마침표(.) ^ \$ * + ? \ | () { } []

- **마침표 (.)** 메타문자 - 해당 위치에 모든(아무) 문자 (1개) 매치 (\n 제외)
- a.b처럼 적혀있으면 정규표현식에서는 a1b 나 aOb나 같은 패턴으로 여겨짐

```
res = re.findall(r"a.b", 'a#b')  
print(res) # ['a#b']
```

- **[^]** : 대괄호 안에 있는 문자들을 제외한 나머지 문자들과 매치시킵니다.
예를 들어서 [^abc]는 a,b,c가 아닌 다른 문자들과 매치하고
[^a-z]은 소문자 알파벳이 아닌 다른 문자들과 일치합니다.

≠

- **^[]** : 문자열의 시작부분이 []대괄호 범주 안의 문자열로 시작되는 문자들과 매치


```
s = '''aaa-xxx  
bbb-yyy  
ccc-zzz'''
```

정규표현의 메타 문자 **^**는 문자열의 앞 머리에 매치된다.

기본적으로는 문자열 전체의 앞머리만 매치하지만,

여러라인 문자열일 경우 re.MULTILINE을 지정하면 각 행의 선두에도 매치되도록 할 수 있음

```
result = re.findall(r"^[a-z]+", s, flags=re.MULTILINE)  
print(result)  
# 출력 ['aaa', 'bbb', 'ccc'] , flags 미지정시 ['aaa']
```

➤ **\$** : 정규표현의 메타 문자 **\$**는 문자열의 맨 마지막에 매치된다.

문자열의 끝 부분(MULTILINE플래그로 개행한 끝 부분도 매치)

```
s = '''python  
regular  
Exp'''
```

```
result = re.findall(r"[a-z]+$", s, flags=re.MULTILINE)
```

```
print(result) # ['python', 'regular', 'xp'] , flags 미지정시 ['xp']
```

➤ **+(메타)** 문자 : 앞의 문자 한개 이상인지 체크

예) + 앞의 문자가 한개 이상있어야 매치

예) + 앞의 문자가 없다면 None(비 매치)

```
src = 'python aac test'
```

```
result = re.search(r"a+c",src) # a문자가 한개 이상있어야 매치
```

```
print(result) # <re.Match object; span=(7, 10), match='aac'>
```

➤ ***(메타)** 문자 : 곱하기 문자 (같은 문자 반복 체크)

예) * 앞의 문자가 0개 이상있어야 매치

예) * 앞의 문자가 없어도 매치됨(0개 이상이기 때문)

```
src = 'python aaaaacademy'
```

```
result = re.search(r"a*c",src) # a문자가 0개 이상있어야 매치
```

```
print(result) # <re.Match object; span=(7, 13), match='aaaaac'>
```

➤ **? 메타문자** : 앞의 문자가 없거나 하나가 있을 경우 찾기
? 앞의 문자가 없거나 하나 있을때 매치, 여러개 올경우 비매치(None)

```
src = 'gdkkllsd'  
result = re.search(r"go?d",src)  
print(result) # gd, god 매치, good 비매치, 'gdkkllsd' ==> 'gd'로 매치
```

➤ **{n, m} 메타문자** : 일정 갯수인 패턴 찾기
{n,m} 앞의 문자열이 n번 이상, m번 이하 나타났다면 맞는 패턴

```
s1 = '휴대전화예요.010-1234-1991'  
m1 = re.search(r"[0-9]{2,3}-[0-9]{4}-[0-9]{4}", s1)  
  
print(m1) # <re.Match object; span=(7, 20), match='010-1234-1991'>  
  
print(m1.group()) # 매치 항목 추출 010-1234-1991
```

➤ **^ 메타** : 시작 문자가 일치하는 경우 찾기

```
s = '안녕하세요. 좋은 아침이에요'
m = re.search(r"^안녕", s)
print(m.group()) # 안녕
```

➤ **\$ 메타** : 마지막 문자가 일치하는 문자열 찾기

```
s = '안녕하세요. 좋은 아침이에요'
m = re.search(r"[ㄱ-힅]+요$", s) # '요'로 끝나는 문자열
print(m.group())# 아침이에요
```

➤ **| 메타** : 여러개 조건 만족하는 패턴 찾기 (or, 또는)

```
s = 'There is a gold and apple'
```

gol*d 랑 ap+le 중 하나만 일치해도, 맞는 패턴

```
m = re.findall(r"gol*d|ap+le", s) # | 앞뒤로 정규식이 2개 있다
```

```
print(m) # ['gold', 'apple']
```

➤ **() 메타**: 그룹으로 묶기

찾은 결과물에서 특정 부분만 빼내고 싶을때

패턴과 일치하는 부분 부분을 추출하기 위해 ()로 그룹핑해줌

```
s = '안녕하세요. 정규 표현식 예제 입니다. 이 예제는 2024-03-12에 작성했어요'
```

```
m = re.search(r"([0-9]{4})-([0-9]{2})-([0-9]{2})", s) # 년,월,일 구분 날짜 찾기
```

```
print(m.group()) # 매칭된 모든 문자 추출됨
```

```
print(m.group(1)) # 2024, 정규식과 일치하는 첫번째 항목 추출
```

```
print(m.group(2)) # 03, 정규식과 일치하는 두번째 항목 추출
```

```
print(m.group(3)) # 12, 정규식과 일치하는 세번째 항목
```

➤ 문자 클래스 [] 메타

[] 안에는 1개 이상의 문자를 넣을 수 있고, 그 문자 중 하나라도 일치한다면
일치하는 패턴으로 판단

```
s = 'There is a gold and apple'
m = re.findall(r"[apd]", s) # 'a', 'p', 'd' 패턴과 와 일치하는 모든 문자 반환
print(m) # ['a', 'd', 'a', 'd', 'a', 'p', 'p']
```

문자 클래스 [] 에서 '-' 는 문자클래스 내에서 범위를 의미
예) [0-9] 는 숫자 문자 범위 패턴

➤ 문자 클래스 [] 안에 '^' 가 들어가면, 그 내용이 반대가 됨

예) [^0-9] 는 숫자 문자 범위 패턴이 아닌 문자

```
s = 'phone number is 1111-3333'
m = re.findall(r"[^0-9]", s)
print(m)
```

출력 : ['p', 'h', 'o', 'n', 'e', ' ', 'n', 'u', 'm', 'b', 'e', 'r', ' ', 'i', 's', ' ', '-']

5.4 로우 문자열(raw string) 표기법

이스케이프 문자열을 표현하기 위하여 'w'(백슬래시)문자를 사용하기 때문에, 문자 'w'를 정규표현식 으로 표현하기 위해서는 '\\\\w'로, 일반 문자열에서는 'ww'로 표현해야 하는 불편함을 해소

➤ 로우 문자열(raw string) 표기법은 문자열 앞에 'r'을 더한 것으로, w(백슬래시) 문자를 이스케이프 문자열로 처리하지 않고 일반 문자와 동일하게 처리 이렇게 함으로써 정규표현식 및 문자열에서 'w'를 간단하게 표현할 수 있음
원시 문자열 표현법(raw string) : r' ' 을 사용하면 편함

```
p = re.compile(r'\\w\\wprogram') # r'\\w\\wprogram' == '\\\\w\\\\wprogram'
res = p.search('python \\w\\wprogram good')
print(res, res.group()) # res.group()항목의 문자열 : \\w\\wprogram
```

```
res = re.search('\\\\w\\\\w\\\\w\\\\w+', '\\w\\wapple') # raw string 표기법 사용 안함
print(res, res.group()) # res.group()항목의 문자열 : \\w\\wapple
```

```
res = re.search(r'\\\\w\\\\w\\\\w+', '\\w\\wapple') # raw string 표기법 사용 함
print(res, res.group()) # res.group()항목의 문자열 : \\w\\wapple
```

5.5 별도 표기법을 가진 문자 클래스

- 문자클래스 중 '[a-z]', '[A-Z]', '[0-9]', ' ' 는 꽤 빈번하게 사용됨
이렇게 빈번하게 사용되는 문자 클래스이기 때문에 별도의 표기법을 제공
- 대표적인 별도 표기 문자 클래스 6가지
 - `Wd` == 숫자 == `[0-9]`
 - `WD` == 숫자가 아닌것 == `[^0-9]`
 - `Ws` == 공백 문자(내려쓰기, 탭 등을 포함) == `[WtWnWr]`
 - `WS` == 공백 문자가 아닌것 == `[^WtWnWr]`
 - `Ww` == 문자 + 숫자 + 언더바(_) == `[a-zA-Z0-9_]`
 - `WW` == 문자 + 숫자 + 언더바(_) 아닌것 == `[^a-zA-Z0-9_]`
- `Ww` 와 `WW`의 문자는 단순 영문뿐 아니라 한글도 포함


```
result = re.findall(r"WW", '나는 Python 정규 Expression을 배우고 있습니다.')
print(result) #문자 + 숫자 + 언더바(_) 아닌것 ==> [' ', ' ', ' ', ' ', ' ', ' ', ' ']
```

```
result = re.findall(r"Wd", 'Python 1991')
print(result) # 숫자문자 한문자 ==> ['1', '9', '9', '1']
```

```
result = re.findall(r"WsWd", '별도 표기 문자 클래스 3991 체크')
print(result) # 공백문자뒤에 숫자문자가 같이 있는 패턴 체크 ==> [' 3']
```

```
result = re.findall(r"[WsWd]", '별도 표기 문자 클래스 3991 체크')
print(result) # 공백문자, 숫자문자를 개별적으로 모두 찾음
# 출력 [' ', ' ', ' ', ' ', ' ', '3', '9', '9', '1', ' ']
```

```
# 이메일 주소 패턴 : '[a-zA-Z]+[_.a-zA-Z0-9]+@[a-z]+[.][a-z]+[.]*[a-z]*'
# * : 앞의 [ ] 문자가 0개 이상일 경우( 없어도 됨 )
mail_address = '홍길동 이메일 주소는 Hong_gilldong@naver.com 입니다'
result = re.findall(r"[a-zA-Z]+[_.a-zA-Z0-9]+@[a-z]+[.][a-z]+[.]*[a-z]*", mail_address)
print(result) # ['Hong_gilldong@naver.com']
```

5.6 숫자 문자 검색 예)

```
import re
```

```
test_str = ""I am Hong gill-dong. I live in Seoul.
```

```
Sample text for testing:
```

```
abcde7fgAvx ABCE8DE
```

```
001213 *^^^#O)#
```

```
23.89 123-567 9,56
```

```
""
```

```
# 0 ~ 9 사이 숫자 검색, 만약 123을 찾으면 1,2,3 을 하나의 독립적 결과로 반환
```

```
pattern1 = re.compile(r"[0-9]")
```

```
res = pattern1.findall(test_str)
```

```
print(res)
```

```
# 하나 이상의 0 ~ 9 사이 숫자 검색, 만약 123을 찾으면 123 하나로 반환
```

```
pattern2 = re.compile(r"[0-9]+")
```

```
res = pattern2.findall(test_str)
```

```
print(res)
```

5.7 정규 표현식 / 영문 대/소문자 검색 예)

```
import re
test_str = ""I am Hong gill-dong. I live in SEoul.
Sample text for testing:
abcde7fgAvx ABCE8DE
001213 *^^#O)#
23.89 123-567 9,56
""
```

```
pattern1 = re.compile(r"[a-z]") # a ~ z 사이의 소문자 검색, 만약 dong을 찾으면 'd', 'o', 'n', 'g' 처럼 독립적
결과반환
```

```
pattern2 = re.compile(r"[a-z]+") # a ~ z 사이 하나 이상의 소문자 검색, Hong이면 'ong'반환
```

```
res = pattern1.findall(test_str)
```

```
print(res)
```

```
res = pattern2.findall(test_str)
```

```
print(res)
```

```
pattern1 = re.compile(r"[A-Z]") # A ~ Z 사이의 대문자 검색, 만약 DE를 찾으면 'D', 'E' 처럼 독립적 결과반
환
```

```
pattern2 = re.compile(r"[A-Z]+") # A ~ Z 사이 하나 이상의 대문자 검색, SEoul이면 'SE'반환
```

```
res = pattern1.findall(test_str)
```

```
print(res)
```

```
res = pattern2.findall(test_str)
```

```
print(res)
```

5.8 정규 표현식 / 영문자 범위, 한글 범위 검색 예)

```
import re
```

```
test_str = """I am Hong gill-dong. I live 강남in SEoul.
```

```
Sample 모든 문자 text for testing:
```

```
abcde7fgAvx ABCE8DE 파이썬
```

```
001213 *^^^#O)#
```

```
23.89 Number:123-567 9,56
```

```
"""
```

```
pattern1 = re.compile(r"[a-zA-Z]") # a ~ z 와 A ~ Z 범위의 모든 영 문자 검색
```

```
res = pattern1.findall(test_str)
```

```
print(res)
```

```
pattern2 = re.compile(r"[a-zA-Z]+") # a ~ z 와 A ~ Z 범위의 하나 이상의 영 문자 검색
```

```
res = pattern2.findall(test_str)
```

```
print(res)
```

```
pattern3 = re.compile(r"[가-힣]+") # 한글 완성형 범위 표현 정규 표현식
```

```
res = pattern3.findall(test_str)
```

```
print(res)
```

5.9 정규 표현식 / 일치된 숫자 패턴 검색 예)

```
import re
```

```
test_num = '아카데미 전화번호 : 031-737-7900'
```

```
# 다양한 숫자문자 패턴
```

```
pattern1 = re.compile(r"[0-9][0-9][0-9]-[0-9][0-9][0-9]-[0-9][0-9][0-9][0-9]")
```

```
res = pattern1.findall(test_num)
```

```
print(res)
```

```
pattern2 = re.compile(r"WdWdWd-WdWdWd-WdWdWdWd") # Wd : 숫자문자
```

```
res = pattern2.findall(test_num)
```

```
print(res)
```

```
# 숫자문자 몇개 반복 : Wd{3} => 숫자3개씩 묶어서 검색 , Ww{3} => 문자3개씩 묶어서 검색
```

```
pattern3 = re.compile(r"Wd{3}-Wd{3}-Wd{4}")
```

```
res = pattern3.findall(test_num)
```

```
print(res)
```

5.10 정규 표현식 / 메타 문자 활용 예)

```
import re
test_str = """I am Hong gill-dong. I live 강남in SEoul.
Sample 모든 문자 python text for testing:
abcde7fgAvx AB#!CE8DE 파이썬
001213 *^^^#O)#
23.89 Number:123-567 9,56
"""
```

```
pattern4 = re.compile(r"[a-zA-Z0-9]+") # 영문대소문자+숫자문자가 하나이상인 문자 검색
res = pattern4.findall(test_str)
print(res)
```

+ : 앞에 나온 패턴이 하나 이상 반복으로 일치하는 것 검색 (결국 해당 패턴이 하나 이상인 것)
* : 앞에 나온 패턴이 없거나, 해당 패턴이 여러개 반복되는 것 검색 (결국 해당 패턴과 유사한 모든것) 
pattern4 = re.compile(r"Ww+") # 문자가 하나 이상인 문자 검색 (특수문자를 제외 시킴)<=Ww(문자) 반복의미
res = pattern4.findall(test_str)
print(res)

```
test_str3 = 'aaaacts abbbbc ac c abc tc aaak'
# + 앞의 'a' 문자가 하나 이상 반복이고 'c'로 연결된 문자패턴
print( re.findall(r"a+c", test_str3) )
# * 앞의 'a' 문자가 여러개 반복되거나 없어도 되는 'c'로 연결된 문자패턴
print( re.findall(r"a*c", test_str3) )
```

['aaaac', 'ac']

['aaaac', 'c', 'ac', 'c', 'c', 'c']

5.11 정규 표현식 / ^ , .. , ? 메타 문자 활용 예)

```
import re
```

```
test_str = """I am Hong gill-dong. I live 강남in SEoul.
```

```
Sample 모든 문자 python text for testing:
```

```
abcde7fgAvx AB#!CE8DE 파이썬 est test
```

```
001213 *^^^#O)#
```

```
23.89 Number:123-567 9,56
```

```
"""
```

^ 가 대괄호 안에 있을 경우 포함하지 않는다는 의미

```
pattern1 = re.compile(r"[^a-z]+") # a ~ z까지 포함되지 않는 것
```

```
res = pattern1.findall(test_str)
```

```
print(res)
```

```
pattern2 = re.compile(r"[^A-Z]+") # A ~ Z까지 포함되지 않는 것
```

```
res = pattern2.findall(test_str)
```

```
print(res)
```

```
pattern3 = re.compile(r"t..t") # t문자문자t 패턴 검색
res = pattern3.findall(test_str)
print(res)
```

? : 앞의 패턴 't' 가 있어도 되고 없어도 됨

```
test_str1 = 'test goodtest badtest est testprog'
```

```
pattern4 = re.compile(r"t?est") # ? 앞의 't' 문자 패턴이 있어도 되고 없어도 됨
res = pattern4.findall(test_str1)
print(res)
```

['test', 'test', 'test', 'est', 'test']

? 와 w+ 조합

```
pattern4 = re.compile(r"t?estWw+") # test 나 est로 시작하는 문자열 뒤에 하나 이상의 문자가 있는 패턴 검색
```

```
res = pattern4.findall(test_str1)
print(res)
```

['testprog']

? 와 w* 조합

```
pattern5 = re.compile(r"t?estWw*") # test 나 est로 시작하는 문자열 뒤에 문자가 없어도 됨으로 해당문  
자열로 시작하는 모든 문자 검색
```

```
res = pattern5.findall(test_str1)
print(res)
```

['test', 'test', 'test', 'est', 'testprog']

5.12 정규 표현식 주요 사용 메서드 활용 예)



```
import re
```

```
str1 = 'te#st python,good study'
```

```
#patterns = re.compile('[Ws,#]')
```

_____ → pattern 변수 대신 직접가능

```
sub_res = re.sub(r"[Ws,#]", "", str1) # 공백문자, 콤마문자, #문자 와 일치하는 항목을 ""로 치환, 즉 삭제
```

print(sub_res) _____ → testpythongoodstudy

```
split_res = re.split(r"[Ws,#]", str1) # 공백문자, 콤마문자, #문자 와 일치하는 항목으로 분리
```

print(split_res) _____ → ['te', 'st', 'python', 'good', 'study']

```
sub_res = re.sub(r"[a-z]", "", str1)
```

a-z 범위 내 모든 문자와 일치하는 항목을 ""로 치환(삭제) (t, e, s, t, p, y, ...)

print(sub_res) _____ → # ,

sub_res = re.sub(r"[a-z]+", "", str1) # a-z 범위 내의 하나 이상의 문자 (te, st, python..)

print(sub_res) → # ,

sub_res = re.sub(r"^[a-z]", "", str1) # a-z 범위 이외의 문자를 찾아 ""로 치환, 즉 삭제

print(sub_res) → testpythongoodstudy

find_res = re.findall(r"ts", str1) # 'ts'와 일치하는 모든 항목을 찾아라!

print(find_res) → [] 현재 일치 항목 없음

find_res = re.findall(r"[ts]", str1) # 't' 또는 's' 와 일치하는 항목을 찾아라!

print(find_res) → ['t', 's', 't', 't', 's', 't']

문제) 다음 문자열을 정규식을 활용해 영문자(대소문자포함)를 제외한 최종 결과의 문자열로 변환
string_exam = '파이썬, library 활용한 Text Preprocessing!!'

결과 : library,Text,Preprocessing

Thank You

